Module 7 – Advanced PHP Excercises practical :-

-BARAD VIPUL

1. E-commerce class hierarchy (Product, Category, Order) – Encapsulation

php

```
<?php
class Product {
    private $name;
    private $price;
    private $category;

    public function __construct($name, $price, $category) {
        $this->name     = $name;
        $this->price    = $price;
        $this->category = $category;
    }

    public function getName()      { return $this->name; }
    public function getPrice()     { return $this->price; }
    public function getCategory()  { return $this->category; }
}

class Category {
```

```
    private $name;
    private $products = [];

    public function __construct($name) {
        $this->name = $name;
    }

    public function addProduct(Product $prod) {
        $this->products[] = $prod;
    }

    public function getName()     { return $this->name; }
    public function getProducts() { return $this->products; }
}

class Order {
    private $id;
    private $products = [];

    public function __construct($id) {
        $this->id = $id;
    }
```

```
    public function addProduct(Product $prod) {
        $this->products[] = $prod;
    }

    public function getTotal() {
        $sum = 0;
        foreach ($this->products as $p) {
            $sum += $p->getPrice();
        }
        return $sum;
    }

    public function getId()       { return $this->id; }
    public function getProducts()  { return $this->products; }
}

// Usage example
$cat = new Category("Electronics");
$p1 = new Product("Laptop", 45000, $cat);
$cat->addProduct($p1);
```

```
$order = new Order(101);
$order->addProduct($p1);

echo "Order ID: " . $order->getId() . "\n";
echo "Total: " . $order->getTotal() . "\n";
?>
```

---

2. Book class with discount method

php

```php
<?php
class Book {
    private $title;
    private $author;
    private $price;

    public function __construct($title, $author, $price) {
        $this->title  = $title;
        $this->author = $author;
        $this->price  = $price;
    }

    public function applyDiscount($percent) {
        $discount = $this->price * ($percent / 100);
        return $this->price - $discount;
    }

    public function getTitle()   { return $this->title; }
    public function getAuthor()  { return $this->author; }
    public function getPrice()   { return $this->price; }
}
```

```
$b = new Book("PHP OOP", "John Doe", 800);
echo "After 20% discount: " . $b->applyDiscount(20) . "\n";
?>
```

---

3. Book objects – multiple instances and formatted display

php

```
<?php
```

```php
include_once "Book.php"; // assuming Book class above

$b1 = new Book("PHP OOP", "John Doe", 800);
$b2 = new Book("MySQL Guide", "Jane Smith", 600);
$b3 = new Book("Web Design", "Alex", 500);

$books = [$b1, $b2, $b3];

foreach ($books as $b) {
    printf("Title: %s | Author: %s | Price: ₹%.2f\n",
        $b->getTitle(), $b->getAuthor(), $b->getPrice()
    );
}
?>
```

---

4. Employee hierarchy with FullTime / PartTime

php

```php
<?php
class Employee {
```

```
    protected $name;
    protected $salary;

    public function __construct($name, $salary) {
        $this->name   = $name;
        $this->salary = $salary;
    }

    public function getName()   { return $this->name; }
    public function getSalary() { return $this->salary; }
}

class FullTimeEmployee extends Employee {
    public function calculateBonus() {
        return $this->salary * 0.2; // 20%
    }
}

class PartTimeEmployee extends Employee {
    public function calculateBonus() {
        return $this->salary * 0.05; // 5%
    }
```

```
}

$fte = new FullTimeEmployee("Alice", 60000);
$pte = new PartTimeEmployee("Bob", 30000);

echo $fte->getName() . " bonus: " . $fte->calculateBonus() .
"\n";
echo $pte->getName() . " bonus: " . $pte->calculateBonus() .
"\n";
?>
```

---

5. Calculator with method overloading via arguments

php

```
<?php
class Calculator {
```

```
    public function calculate($a, $b, $op) {
        switch ($op) {
            case 'add':    return $a + $b;
            case 'sub':    return $a - $b;
            case 'mul':    return $a * $b;
            default:       throw new Exception("Invalid operation");
        }
    }
}

$calc = new Calculator();
echo "Add: "  . $calc->calculate(10, 5, 'add') . "\n";
echo "Sub: "  . $calc->calculate(10, 5, 'sub') . "\n";
echo "Mul: "  . $calc->calculate(10, 5, 'mul') . "\n";
?>
```

---

6. PaymentInterface with implementations

php

```
<?php
interface PaymentInterface {
    public function processPayment(float $amount): bool;
```

```
    public function refund(float $amount): bool;
}

class CreditCardPayment implements PaymentInterface {
    public function processPayment(float $amount): bool {
        echo "CREDIT CARD: ₹$amount paid.\n";
        return true;
    }

    public function refund(float $amount): bool {
        echo "CREDIT CARD refund: ₹$amount.\n";
        return true;
    }
}

class PaypalPayment implements PaymentInterface {
    public function processPayment(float $amount): bool {
        echo "PAYPAL: ₹$amount paid.\n";
        return true;
    }

    public function refund(float $amount): bool {
```

```
        echo "PAYPAL refund: ₹$amount.\n";
        return true;
    }
}

$cc = new CreditCardPayment();
$pp = new PaypalPayment();

$cc->processPayment(1000);
$pp->processPayment(500);
?>
```

---

7. Student class with constructor

php

```
<?php
class Student {
    private $name;
    private $age;
```

```
    private $grade;

    public function __construct($name, $age, $grade) {
        $this->name  = $name;
        $this->age   = $age;
        $this->grade = $grade;
    }

    public function display() {
        printf("Name: %s | Age: %d | Grade: %s\n",
            $this->name, $this->age, $this->grade
        );
    }
}

$s = new Student("Rahul", 18, "A+");
$s->display();
?>
```

---

8. Database class with destructor

php

```
<?php
class Database {
    private $conn;
```

```
    public function __construct($host, $user, $pass, $db) {
        $this->conn = new mysqli($host, $user, $pass, $db);
        if ($this->conn->connect_error) {
            throw new Exception("Connection failed: " . $this-
>conn->connect_error);
        }
    }

    public function query($sql) {
        return $this->conn->query($sql);
    }

    public function __destruct() {
        if ($this->conn) {
            $this->conn->close();
            echo "Connection closed.\n";
        }
    }
}

$db = new Database("localhost", "root", "", "test");
$result = $db->query("SELECT 1");
```

?>

---

9. Class using _*set() and* _get() magic methods

php

```
<?php
class DynamicProps {
    private $data = [];
```

```
    public function __set($key, $value) {
        $this->data[$key] = $value;
        echo "Set: $key = $value\n";
    }

    public function __get($key) {
        return $this->data[$key] ?? null;
    }
}

$obj = new DynamicProps();
$obj->name  = "Alice";
$obj->email = "alice@example.com";

echo $obj->name . " – " . $obj->email . "\n";
?>
```

---

10. Static counter with scope resolution

php

```
<?php
class Counter {
    private static $count = 0;
```

```
    public function __construct() {
        self::$count++;
    }

    public static function getCount() {
        return self::$count;
    }
}

new Counter();
new Counter();
new Counter();

echo "Total instances: " . Counter::getCount() . "\n";
?>
```

---

11. Logger and Notifier traits

php

```
<?php
trait Logger {
    public function log($message) {
```

```
        echo "[LOG] " . date("Y-m-d H:i:s") . " – $message\n";
    }
}

trait Notifier {
    public function notify($msg) {
        echo "[NOTIFY] $msg\n";
    }
}

class User {
    use Logger, Notifier;

    private $name;

    public function __construct($name) {
        $this->name = $name;
    }

    public function login() {
        $this->log("User {$this->name} logged in.");
        $this->notify("Hello, {$this->name}!");
```

```
    }
}

$u = new User("Raj");
$u->login();
?>
```

---

12. Account with public/private/protected visibility

php

```
<?php
class Account {
    public $username;
    private $password;
```

```
    protected $accountBalance;

    public function __construct($username, $password,
$balance) {
        $this->username       = $username;
        $this->password       = $password;
        $this->accountBalance = $balance;
    }

    public function setPassword($pass) {
        $this->password = $pass;
    }

    public function getBalance() {
        return $this->accountBalance;
    }

    public function getPassword() {
        return $this->password;
    }
}
```

```
class BankAccount extends Account {
    public function showProtected() {
        return "Protected balance: " . $this->accountBalance .
"\n";
    }
}

$acc = new BankAccount("raj123", "secure123", 50000);
echo $acc->showProtected();
?>
```

---

13. Order with type-hinted array of products

php

```
<?php
class Product {
    private $name;
    private $price;
```

```
    public function __construct($name, $price) {
        $this->name  = $name;
        $this->price = $price;
    }

    public function getPrice() {
        return $this->price;
    }
}

class Order {
    public function calculateTotal(array $products): float {
        $total = 0;
        foreach ($products as $p) {
            $total += $p->getPrice();
        }
        return $total;
    }
}

$p1 = new Product("Laptop", 45000);
```

```
$p2 = new Product("Mouse", 500);


$order = new Order();
echo "Total: " . $order->calculateTotal([$p1, $p2]) . "\n";
?>
```

---

14. Final class Dog (restricting inheritance)

php

```
<?php
class Animal {
    public function speak() {
        echo "Animal makes a sound.\n";
```

```
    }
}

final class Dog extends Animal {
    public function speak() {
        echo "Dog barks.\n";
    }
}

$d = new Dog();
$d->speak();
?>
```

---

15. Email validation + exception

php

```
<?php
function sanitizeEmail($email) {
    $email = trim($email);
    $email = filter_var($email, FILTER_SANITIZE_EMAIL);
```

```
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new InvalidArgumentException("Invalid email format.");
    }

    return $email;
}

try {
    $clean = sanitizeEmail(" user@example.com ");
    echo "Valid email: $clean\n";
} catch (InvalidArgumentException $e) {
    echo "Error: " . $e->getMessage() . "\n";
}
?>
```

---

16. File upload and read with error handling

php

```
<?php
if ($_FILES) {
    $tmp = $_FILES['file']['tmp_name'];
```

```
    $name = $_FILES['file']['name'];

    try {
        if (!file_exists($tmp)) {
            throw new Exception("File not uploaded properly.");
        }

        if (!is_readable($tmp)) {
            throw new Exception("Cannot read uploaded file.");
        }

        $content = file_get_contents($tmp);
        echo "<h3>File: $name</h3><pre>" .
htmlspecialchars($content) . "</pre>";
    } catch (Exception $e) {
        echo "Error: " . $e->getMessage() . "\n";
    }
}
?>

<form method="POST" enctype="multipart/form-data">
    <input type="file" name="file">
```

```
    <button type="submit">Upload & Read</button>
</form>
```

---

17. Send welcome email (basic logic)

php

```
<?php
function sendWelcomeEmail($to, $name) {
    if (!filter_var($to, FILTER_VALIDATE_EMAIL)) {
        return false;
```

```
    }

    $subject = "Welcome to Our System";
    $body    = "Hi $name,\n\nWelcome to our platform!";

    return mail($to, $subject, wordwrap($body, 70));
}

// Example:
if (sendWelcomeEmail("user@example.com", "Raj")) {
    echo "Welcome email sent.\n";
} else {
    echo "Email failed.\n";
}
?>
```

---

18. MVC for user profiles (simple structure)

Model – User.php

php

```
<?php
class User {
    private $id;
```

```
private $name;
private $email;

public function __construct($id, $name, $email) {
    $this->id    = $id;
    $this->name  = $name;
    $this->email = $email;
}

public static function fetchAll() {
    return [
        new User(1, "Raj", "raj@example.com"),
        new User(2, "Priya", "priya@example.com"),
    ];
}

public function toArray() {
    return [
        'id'    => $this->id,
        'name'  => $this->name,
        'email' => $this->email,
    ];
```

```
    }
}
?>
```

View – user_list.php

xml

```
<h2>Users</h2>
<table border="1">
<tr><th>ID</th><th>Name</th><th>Email</th></tr>
<?php foreach ($users as $u): ?>
<tr>
    <td><?= $u['id'] ?></td>
    <td><?= $u['name'] ?></td>
    <td><?= $u['email'] ?></td>
</tr>
<?php endforeach; ?>
</table>
```

Controller – index.php

php

```
<?php
include "User.php";

$users = [];
```

```
$data  = User::fetchAll();
foreach ($data as $u) {
    $users[] = $u->toArray();
}

include "user_list.php";
?>
```

---

19. Library system (User, Book, Transaction)

php

```
<?php
class Book {
    private $id;
    private $title;
```

```
    private $author;

    public function __construct($id, $title, $author) {
        $this->id      = $id;
        $this->title   = $title;
        $this->author  = $author;
    }

    public function getId()    { return $this->id; }
    public function getTitle() { return $this->title; }
    public function getAuthor(){ return $this->author; }
}

class User {
    private $id;
    private $name;

    public function __construct($id, $name) {
        $this->id   = $id;
        $this->name = $name;
    }
```

```
    public function getName() { return $this->name; }
}

class Transaction {
    private $user;
    private $book;
    private $date;

    public function __construct(User $user, Book $book) {
        $this->user = $user;
        $this->book = $book;
        $this->date = date("Y-m-d");
    }

    public function printReceipt() {
        echo "Transaction:\n";
        echo "User: "  . $this->user->getName() . "\n";
        echo "Book: "  . $this->book->getTitle() . " by " . $this-
>book->getAuthor() . "\n";
        echo "Date: "  . $this->date . "\n";
    }
}
```

```
$b = new Book(1, "PHP OOP", "John Doe");
$u = new User(101, "Raj");
$t = new Transaction($u, $b);
$t->printReceipt();
?>
```

---

20. Database class to fetch user data

php

```
<?php
class Database {
    private $conn;
```

```
    public function __construct($host, $user, $pass, $db) {
        $this->conn = new mysqli($host, $user, $pass, $db);
        if ($this->conn->connect_error) {
            throw new Exception("Connection failed: " . $this-
>conn->connect_error);
        }
    }

    public function fetchUsers() {
        $res = $this->conn->query("SELECT * FROM users");
        $users = [];
        while ($row = $res->fetch_assoc()) {
            $users[] = $row;
        }
        return $users;
    }

    public function __destruct() {
        if ($this->conn) {
            $this->conn->close();
        }
    }
```

```
}

$db = new Database("localhost
```